Polymorphism → one same instruction but diff objects understand it differently, and do diff actions.

```
mom.getRedLabel()
dad.getRedLabel()
```

diff obj is calling so behaviour changes of the instruction depending which object is calling.

one same instruction but diff objects understand it differently and do different actions.

Virtual word infront of a function → means every child class can change the function of the parent class (keeping the name same). So it automatically overrides the parent's version.

↓

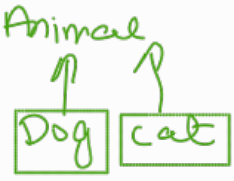
written in parent class only

IMP

Pointers allow one variable to point to diff objects (Dog, Cat, Snake)

A normal object cannot change its type, But a pointer can point to any derived object.

We create a parent pointer to switch b/w different child objects. The parent class stays normal.



Animal a;
Dog d;
cat c;

Animal a = d;
d object is no longer a Dog
it becomes parent because
'a' only keeps the animal part of d

Animal *a; → The pointer belongs to Parent type but it can point to any child object

a = new Dog() Points to dog
a = new Cat()
a = new Snake()

→ now we can point to diff child classes and this is required for polymorphism.

↓
here a doesn't become a Snake
it only points to a Snake

animal a;
dog d;
d = a; (X)

d → expects all dog-specific data

a → has only animal part.

Parent object cannot become a child object.

You can point a parent → child but not
Child → parent

For polymorphism to work there must be virtual in Base class